FOOD AI PLATFORM

Game Design Document & System Architecture

A Hybrid Self-Improving AI System for Budget-Optimized Food Discovery

Next.js · FastAPI · LangChain · LangGraph · PostgreSQL · FAISS

Version 1.0 · April 2026

IDEA VALIDATION REPORT

71 / 100

Overall Viability Score

Dimension	Score	Assessment
Problem Validity	18 / 20	Real gap in emerging-market food discovery
Technical Architecture	14 / 20	Good bones, over-engineered for stage 1
Feasibility	12 / 20	Cold-start & scraping are serious risks
Differentiation	12 / 20	Confidence scoring is a genuine USP
Completeness	15 / 20	Missing monetization + cold-start plan

Strengths

- Budget-constrained food discovery in Islamabad/Pakistan is genuinely underserved — Zomato and Talabat don't optimize for '150 PKR meals'.
- Three-layer trust hierarchy (DB → Internet → Learning) is the correct RAG pattern for production systems.
- Confidence scoring (HIGH / MEDIUM / LOW) shown to users is a UX differentiator that builds trust.
- LangGraph-based orchestration gives you conditional agent routing out of the box.

Risks & Gaps

- Cold-start problem: the vector DB and learning system both need seed data. Who enters the first 500 food items?
- Playwright scraping is legally and technically fragile — consider Tavily/SerpAPI for v1.
- 8 agents is too many for v1. Start with 3: Intent Analyzer, DB Retriever, Response Generator.
- Knapsack optimization is overkill for a ≤150 PKR budget with ~10 candidate items. Use greedy.
- No monetization strategy mentioned — needed before Phase 5+.

1. PRODUCT VISION & PROBLEM STATEMENT

Vision

A budget-first food discovery platform powered by a hybrid AI system that combines a verified knowledge base, live internet intelligence, and continuous user-driven learning. Users enter a budget (e.g. 150 PKR in Islamabad) and receive optimized meal recommendations with full transparency into data confidence.

The Problem

Mainstream food delivery apps like Foodpanda, Cheetay, and Talabat optimize for restaurant revenue, not user budgets. A user asking 'what can I eat for 150 PKR near F-7?' gets no answer from any existing app. Prices change daily. Data goes stale. New dhabas open without any online presence. This platform solves all three.

Core User Promise

Input	150 PKR, Islamabad, F-7 Markaz
Output	Top 5 individual items + 3 optimized meal combos
Transparency	Each result labeled HIGH / MEDIUM / LOW confidence
Learning	User corrections improve future results

Target Users

- Students in Islamabad/Lahore/Karachi managing tight food budgets
- Daily wage workers seeking maximum nutrition per rupee
- Travelers exploring local food without overpaying
- Budget-conscious families comparing dhabas and fast-food outlets

2. HYBRID INTELLIGENCE ARCHITECTURE

The system is built on three intelligence layers with a strict trust hierarchy. No layer is accessed unnecessarily — each gate is conditional.

Layer	Color	Technologies	Trust Level	Purpose
Layer 1 Trusted KB	GREEN	PostgreSQL + FAISS	HIGH (0.75–1.0)	Primary source of truth — verified foods, prices, locat
Layer 2 Internet	BLUE	LangChain Tools Tavily / SerpAPI Playwright	LOW–MED (0.3–0.65)	Fills gaps when DB has no match; unstructured, less
Layer 3 Learning	AMBER	Feedback API Celery Tasks Validation Pipeline	Builds trust	Upgrades internet data to DB via user validation + co

Decision Rules (Priority Order)

1	Always use DB data if available and confidence ≥ 0.6
2	If DB is empty for this query → trigger internet search
3	If both DB + internet have the item → compare prices
4	If price delta > 20% → flag conflict, show range
5	Assign confidence score using formula (see Section 9)
6	Internet data with ≥ 3 user confirmations → promote to DB

3. FULL SYSTEM ARCHITECTURE

Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14 (App Router)	Budget input UI, results display, confidence labels, transparency panel
API Gateway	FastAPI (Python 3.11+)	REST endpoints, request validation, auth, rate limiting
Validation	Pydantic v2 (backend) Zod v3 (frontend)	Schema enforcement at both ends of the wire
Orchestration	LangGraph	Stateful agent workflow with conditional branching
AI Agents	LangChain	Intent parsing, vector retrieval, internet search, scoring
Primary DB	PostgreSQL 16	Restaurants, food items, prices, feedback, confidence data
Vector DB	FAISS (local) / Pinecone (cloud)	Semantic similarity search over food item embeddings
Cache / Queue	Redis 7	Query result cache, Celery task queue
Background Jobs	Celery 5	Scraping tasks, FAISS reindex, DB promotion pipeline
Scraping	Playwright + Chromium	Headless browser automation for menu extraction
Search API	Tavily / SerpAPI	Reliable internet search fallback (use before Playwright)
Monitoring	Prometheus + Grafana	Latency, accuracy, failure rate, agent performance
Tracing	OpenTelemetry	Distributed trace across agent hops
Deployment	Docker Compose → Kubernetes	Local dev → production scaling

Request Flow (Summary)

01. User enters budget + location in Next.js UI
02. Next.js calls POST /api/v1/query on FastAPI
03. FastAPI validates with Pydantic, rate-limits, then invokes LangGraph
04. LangGraph: Intent Agent parses budget/location/cuisine preferences
05. LangGraph: DB Retrieval Agent queries PostgreSQL + FAISS (semantic search)
06. If DB results insufficient → Internet Search Agent calls Tavily / SerpAPI
07. Validation Agent compares DB vs internet prices, detects conflicts
08. Confidence Scorer assigns HIGH/MEDIUM/LOW to each result
09. Budget Optimizer builds meal combos (greedy algorithm, max 3 items)
10. Response Generator formats final JSON response
11. FastAPI returns QueryResponse → Next.js renders results with confidence badges
12. Feedback stored asynchronously → Celery promotion task runs in background

4. API DESIGN

Endpoint Reference

Method	Path	Description	Auth
POST	/api/v1/query	Main query: budget + location → food results	Optional
GET	/api/v1/foods	Filtered food list (location, max_price, confidence_min)	No
POST	/api/v1/feedback	User submits price correction or rating	Optional
GET	/api/v1/health	System health: DB, Redis, FAISS status	No
GET	/api/v1/metrics	Prometheus metrics endpoint	Internal

POST /api/v1/query — Request Schema (Pydantic + Zod)

Both backend (Pydantic) and frontend (Zod) validate against the same contract. Use openapi-typescript to auto-generate Zod schemas from FastAPI's OpenAPI spec.

```
# Backend: Pydantic v2
class QueryRequest(BaseModel):
budget: float = Field(..., gt=0, le=10000) # PKR
location: str = Field(..., min_length=2, max_length=100)
cuisine: Optional[str] = Field(None, max_length=50)
meal_type: Optional[Literal["breakfast","lunch","dinner","snack"]] = None
num_results: int = Field(default=5, ge=1, le=20)

# Frontend: Zod v3
const QueryRequestSchema = z.object({
budget: z.number().positive().max(10000),
location: z.string().min(2).max(100),
cuisine: z.string().max(50).optional(),
meal_type: z.enum(["breakfast","lunch","dinner","snack"]).optional(),
num_results: z.number().int().min(1).max(20).default(5),
})
```

Response Schema

```
class FoodResult(BaseModel):
id: str
name: str
price: float
restaurant: str
location: str
```

```
confidence: Literal["HIGH", "MEDIUM", "LOW"]
confidence_score: float # 0.0 – 1.0
source: Literal["DB", "INTERNET", "HYBRID"]
price_range: Optional[tuple[float, float]] # If conflict

class QueryResponse(BaseModel):
results: list[FoodResult]
combos: list[list[FoodResult]] # Optimized meal combos
total_latency_ms: int
agent_trace: Optional[list[str]] # Debug: which agents ran
```

5. DATABASE DESIGN

PostgreSQL Schema

```
CREATE TABLE restaurants (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
name TEXT NOT NULL,
location TEXT NOT NULL,
city TEXT NOT NULL DEFAULT 'Islamabad',
latitude DECIMAL(9,6), longitude DECIMAL(9,6),
rating DECIMAL(2,1),
source TEXT CHECK (source IN ('MANUAL','SCRAPED','USER')),
verified BOOLEAN DEFAULT FALSE,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE food_items (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
restaurant_id UUID REFERENCES restaurants(id) ON DELETE CASCADE,
name TEXT NOT NULL,
name_embedding VECTOR(1536), -- pgvector / FAISS
category TEXT, -- 'biryani', 'burger', 'paratha'
price DECIMAL(8,2) NOT NULL,
price_min DECIMAL(8,2), -- Range when conflict detected
price_max DECIMAL(8,2),
currency TEXT DEFAULT 'PKR',
confidence TEXT CHECK (confidence IN ('HIGH','MEDIUM','LOW')),
confidence_score DECIMAL(4,3),
source TEXT CHECK (source IN ('DB','INTERNET','HYBRID')),
last_verified TIMESTAMPTZ,
verified_count INTEGER DEFAULT 0
);

CREATE TABLE feedback (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
food_id UUID REFERENCES food_items(id),
query_id TEXT,
rating SMALLINT CHECK (rating BETWEEN 1 AND 5),
price_correct BOOLEAN,
actual_price DECIMAL(8,2),
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Key indexes
CREATE INDEX idx_food_price ON food_items (price);
CREATE INDEX idx_food_category ON food_items (category);
CREATE INDEX idx_food_score ON food_items (confidence_score DESC);
```

Embedding Design

Each food item's name is embedded using OpenAI text-embedding-3-small (1536 dimensions). Embeddings are stored in FAISS for local dev and optionally in Pinecone for production. The FAISS index is rebuilt nightly via a Celery task (reindex_task.py). Semantic search enables queries like 'cheap rice dish' to match 'Chicken Biryani' even without keyword overlap.

6. AGENT DESIGN

Agents are implemented as LangChain Runnable components and composed via LangGraph. For v1, start with 3 agents only. Expand to the full set in phases 4–6.

Intent Analyzer — Phase v1

- Input: raw user query string + budget float
- Extracts: budget (normalized), city, area, cuisine preference, meal_type
- Uses: LLM with structured output (Pydantic model)
- Output: IntentResult TypedDict fed into AgentState

Vector Retrieval Agent — Phase v1

- Embeds the intent query using text-embedding-3-small
- Runs FAISS similarity search (top-k=20)
- Filters by price <= budget and location match
- Sets needs_internet=True if results < 3

Response Generator — Phase v1

- Takes validated + scored results from state
- Formats FoodResult list + combo list
- Adds human-readable explanation per result
- Returns final QueryResponse

Internet Search Agent — Phase v2

- Called only when needs_internet=True
- Uses Tavily Search API with location-scoped queries
- Falls back to SerpAPI if Tavily quota exhausted
- Raw results fed to Validation Agent before use

Data Validation Agent — Phase v2

- Compares DB price vs internet price for same item
- Flags conflict if delta > 20%
- Sets price_range on conflicted items
- Triggers promotion_task if internet data is highly reliable

Confidence Scoring Agent — Phase v2

- Runs compute_confidence() formula on each item
- Factors: source, days_since_verified, user_rating_count, price_conflict
- Assigns HIGH/MEDIUM/LOW label + 0.0–1.0 score

Budget Optimizer — Phase v2

- Greedy combination builder (not knapsack)
- Generates all item combos up to max_items=3 within budget
- Ranks by: most items first, then highest avg confidence
- Returns top 5 combos

Learning Agent — Phase v3

- Processes accumulated feedback records nightly via Celery

- If actual_price != stored_price with confidence: updates food_items
- If internet item has >=3 confirmations: promotes to DB source
- Triggers FAISS reindex after DB updates

7. LANGGRAPH WORKFLOW

AgentState (TypedDict)

```
class AgentState(TypedDict):
# ■■ Input ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
budget: float
location: str
cuisine: Optional[str]

# ■■ Intermediate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
intent: dict # From IntentAnalyzer
db_results: List[FoodResult]
internet_results:List[dict] # Raw, unvalidated
conflicts: List[dict] # Price disagreements
needs_internet: bool # Routing flag

# ■■ Output ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
final_results: List[FoodResult]
combos: List[List[FoodResult]]
```

Graph Definition

```
def build_graph() -> CompiledGraph:
g = StateGraph(AgentState)

# Register nodes
g.add_node("intent", analyze_intent)
g.add_node("db_retrieve", retrieve_from_db)
g.add_node("internet_search", search_internet)
g.add_node("validate", validate_and_score)
g.add_node("optimize", optimize_budget)

g.set_entry_point("intent")
g.add_edge("intent", "db_retrieve")

# Conditional: only call internet if DB results < threshold
g.add_conditional_edges(
"db_retrieve",
lambda s: "internet_search" if s["needs_internet"] else "validate",
{"internet_search": "internet_search", "validate": "validate"}
)
g.add_edge("internet_search", "validate")
g.add_edge("validate", "optimize")
g.add_edge("optimize", END)
```

```
return g.compile()
```

Node Responsibilities

Node	Input Keys Read	Output Keys Written
analyze_intent	budget, location, cuisine	intent
retrieve_from_db	intent	db_results, needs_internet
internet_search	intent	internet_results
validate_and_score	db_results, internet_results	final_results, conflicts
optimize_budget	final_results, budget	combos

8. CONFIDENCE SCORING & DECISION ENGINE

Confidence Formula

Each food result receives a numeric score (0.0–1.0) and a categorical label. The formula accounts for data source, recency, user validation, and price conflicts.

```
def compute_confidence(
source: str, # 'DB' | 'INTERNET'
days_since_verified: int,
user_rating_count: int,
price_conflict: bool,
scrape_reliability: float, # 0.0–1.0 per source site
) -> tuple[float, str]:

if source == 'DB':
base = 0.85
recency_penalty = min(0.30, days_since_verified * 0.01)
rating_bonus = min(0.10, user_rating_count * 0.005)
conflict_penalty = 0.15 if price_conflict else 0.0
score = base + rating_bonus - recency_penalty - conflict_penalty

elif source == 'INTERNET':
base = 0.45
score = base * scrape_reliability

score = round(max(0.0, min(1.0, score)), 3)
label = 'HIGH' if score >= 0.75 else ('MEDIUM' if score >= 0.45 else 'LOW')
return score, label
```

Score Interpretation

Label	Score Range	Meaning	UI Display
HIGH	0.75 – 1.0	DB-verified, recently confirmed by users	Green badge
MEDIUM	0.45 – 0.74	DB data or recent reliable internet match	Amber badge
LOW	0.0 – 0.44	Unverified external data, stale DB entry	Red badge + disclaimer

Conflict Resolution

When the same food item appears in both DB and internet with different prices:

- Calculate price_delta = abs(db_price - internet_price) / db_price
- If delta <= 20%: trust DB price, assign MEDIUM confidence
- If delta > 20%: flag conflict, set price_range = (min, max), show to user

- Log conflict to internet_cache for human review queue
- After 3 user confirmations of internet price → update DB + re-score

9. BUDGET OPTIMIZATION ENGINE

Why Not Knapsack?

Algorithm Choice Rationale

- For a 150 PKR budget with ~10–20 candidate items and max 3 items per combo, brute-force combinations are O(n choose k) = O(20 choose 3) = 1140 iterations — trivially fast in microseconds.
- The 0/1 Knapsack algorithm provides identical results but adds O(n*W) DP complexity and code maintenance overhead with zero practical benefit at this scale.
- Knapsack becomes worth it only when: budget is very high, item pool > 500, or fractional items (buffet pricing) are introduced.

Greedy Combo Builder

```
from itertools import combinations

def build_combos(
items: list[FoodResult],
budget: float,
max_items: int = 3,
max_combos: int = 5,
) -> list[list[FoodResult]]:
"""
Generate all valid combos within budget, rank by value.
Returns top max_combos results.
"""
valid = []
for size in range(1, max_items + 1):
for combo in combinations(items, size):
total = sum(item.price for item in combo)
if total <= budget:
valid.append((list(combo), total))

# Sort: most items first (value), then highest avg confidence
valid.sort(key=lambda x: (
-len(x[0]),
-sum(i.confidence_score for i in x[0]) / len(x[0])
))
return [c[0] for c in valid[:max_combos]]
```

Example Output for 150 PKR Budget

Combo	Items	Total Cost	Confidence
#1 (Best Value)	Aloo Paratha + Chai + Chutney	145 PKR	HIGH, HIGH, MED
#2	Egg Paratha + Chai	130 PKR	HIGH, HIGH
#3	Chicken Karahi (half)	150 PKR	MED
#4	Daal + Roti + Lassi	140 PKR	HIGH, HIGH, LOW
#5 (Single item)	Chicken Biryani (plate)	120 PKR	HIGH

10. PROJECT FOLDER STRUCTURE

Monorepo Layout

```
food-ai-platform/
■■■ frontend/ # Next.js 14 (App Router)
■ ■■■ app/
■ ■ ■■■ page.tsx # Budget input landing page
■ ■ ■■■ results/page.tsx # Results with confidence UI
■ ■ ■■■ api/query/route.ts # Thin proxy to FastAPI
■ ■■■ components/
■ ■ ■■■ BudgetInput.tsx
■ ■ ■■■ FoodCard.tsx # Confidence badge included
■ ■ ■■■ ConfidenceBadge.tsx # HIGH / MEDIUM / LOW
■ ■ ■■■ MealCombo.tsx
■ ■ ■■■ TransparencyPanel.tsx
■ ■■■ lib/api.ts # Typed fetch wrapper
■ ■■■ lib/validators.ts # Zod schemas
■
■■■ backend/ # FastAPI (Python 3.11+)
■ ■■■ app/
■ ■■■ main.py # App init, CORS, middleware
■ ■■■ config.py # Pydantic Settings
■ ■■■ api/v1/ # Route handlers
■ ■ ■■■ query.py # POST /query
■ ■ ■■■ foods.py # GET /foods
■ ■ ■■■ feedback.py # POST /feedback
■ ■ ■■■ health.py # GET /health
■ ■■■ agents/ # LangChain agent modules
■ ■ ■■■ intent_analyzer.py
■ ■ ■■■ vector_retrieval.py
■ ■ ■■■ internet_search.py
■ ■ ■■■ data_validator.py
■ ■ ■■■ confidence_scorer.py
■ ■ ■■■ budget_optimizer.py
■ ■ ■■■ learning_agent.py
■ ■■■ graph/ # LangGraph
■ ■ ■■■ state.py # AgentState TypedDict
■ ■ ■■■ workflow.py # Graph build + compile
■ ■ ■■■ nodes.py # Node functions
■ ■■■ db/ # SQLAlchemy + Alembic
■ ■ ■■■ session.py
■ ■ ■■■ models.py
■ ■ ■■■ migrations/
■ ■■■ vector/ # FAISS wrapper
■ ■ ■■■ store.py
■ ■ ■■■ embedder.py
■ ■■■ scraper/ # Playwright
■ ■ ■■■ playwright_runner.py
■ ■ ■■■ extractors/
■ ■■■ tasks/ # Celery
■ ■ ■■■ celery_app.py
■ ■ ■■■ scrape_task.py
■ ■ ■■■ reindex_task.py
■ ■ ■■■ promotion_task.py
■ ■■■ schemas/ # Pydantic models
■ ■■■ core/ # Confidence, optimizer, errors
■
■■■ infra/
■ ■■■ docker-compose.yml
■ ■■■ nginx.conf
```

■■■ .env.example

11. INTERNET AGENT & SCRAPING PIPELINE

Search Strategy (Priority Order)

1. Tavily Search API

Primary internet source. Handles anti-bot, gives clean structured results. Use for general food price queries.

2. SerpAPI

Fallback to Tavily. Good for Google Maps / local business data. Rate-limit aware.

3. Playwright Scraper

Last resort. Only for specific, stable menu pages (e.g. restaurant's own website). Never scrape aggregators like Foodpanda directly.

4. Manual Data Entry

Most reliable internet-to-DB path. Build an admin panel for this in Phase 1.

Playwright Scraping Pipeline

```
async def scrape_menu(url: str, extractor: str) -> list[dict]:
async with async_playwright() as p:
browser = await p.chromium.launch(headless=True)
page = await browser.new_page()
await page.goto(url, wait_until='networkidle')

# Run site-specific extractor
items = await EXTRACTORS[extractor](page)

await browser.close()
return items

# Each extractor returns normalized dicts:
# { name, price, currency, restaurant, location, scraped_at }
```

Source Reliability Scoring

Source	Base Reliability	Notes
Restaurant's own website	0.85	Direct source, rarely changes menu pricing
Tavily Search API	0.70	Aggregated but validated
SerpAPI / Google Maps	0.65	Often has stale pricing
Food aggregator scrape	0.40	Commission-adjusted prices, frequent changes
Forum / social media	0.20	Unverified user reports

Scraping Legal & Technical Risks

- Always check robots.txt before scraping any site.
- Foodpanda, Daraz, and similar aggregators explicitly prohibit scraping in their ToS.
- Use Tavily/SerpAPI as the primary internet layer — only build Playwright scrapers for restaurant sites that permit it.
- Rotate user agents and add random delays (1–3s) to avoid rate-limiting.
- Store raw scraped data in internet_cache with an expires_at TTL of 24 hours.

12. OBSERVABILITY & DEVELOPMENT ROADMAP

Metrics to Track (Prometheus)

Metric	Type	Alert Threshold
query_latency_ms	Histogram	> 3000ms p95
db_hit_rate	Gauge	< 60% (too much internet fallback)
internet_fallback_rate	Counter	> 40% of queries
confidence_score_avg	Gauge	< 0.55 (data quality degrading)
feedback_submission_rate	Counter	< 2/day (learning stalling)
scrape_failure_rate	Counter	> 15% per source
agent_error_rate	Counter	> 1% (LangGraph node failures)
promotion_queue_length	Gauge	> 100 (backlog building)

Development Roadmap

Phase	Duration	Deliverable	Success Criteria
1 — Static MVP	2 weeks	Next.js UI + FastAPI skeleton + hardcoded data	10 users can query
2 — DB Layer	2 weeks	PostgreSQL + Alembic + 200 seed foods + ingredients	DB hit rate > 80%
3 — Vector Search	1 week	FAISS + semantic search enabled	Semantic queries work
4 — Core Agents	3 weeks	Intent + Retrieval + Response (LangGraph)	p95 latency < 2s
5 — Internet Layer	2 weeks	Tavily + SerpAPI fallback	Coverage gap < 20%
6 — Scraping	2 weeks	Playwright for 3 trusted restaurant sites	3 sources scraped
7 — Learning Loop	2 weeks	Feedback API + Celery promotion pipeline	Score improves 5%/wk
8 — Monitoring	1 week	Prometheus + Grafana + alert rules	All metrics tracked

13. CHALLENGES, RISKS & EVALUATION METRICS

Key Challenges & Mitigations

Challenge	Risk Level	Mitigation
Cold start — no seed data	HIGH	Manual entry of 200+ items before launch. Build admin panel in Phase 1.
Scraping ToS violations	HIGH	Use Tavily/SerpAPI. Only scrape sites with explicit permission.
Price staleness	MEDIUM	Store verified_at timestamp. Auto-downgrade confidence after 30 days.
LLM latency (>3s)	MEDIUM	Cache frequent queries in Redis (TTL=1hr). Stream responses.
FAISS index drift	MEDIUM	Nightly Celery reindex_task. Monitor embedding freshness metric.
Over-engineering	MEDIUM	Ship Phase 1 to 10 real users before building Phase 4+.
Urdu/mixed-language queries	LOW	Use multilingual embedding model (multilingual-e5-large).

Evaluation Metrics

Metric	Formula / Definition	Target
DB Hit Rate	DB queries / total queries	> 70%
Result Relevance	User thumbs-up / total results shown	> 75%
Price Accuracy	1 - |actual - predicted| / actual (avg)	> 85%
Query Latency p95	95th percentile response time	< 2500ms
Confidence Calibration	% HIGH results that users confirm correct	> 90%
Learning Rate	Confidence score improvement per week	> +2% / week
Cold Start Coverage	% of Islamabad queries with >=3 results	> 80% in week 2

Final Recommendations

Solve cold start first. Manually enter 200+ Islamabad food items before writing a single agent. AI quality is capped by data quality.

Tavily before Playwright. Tavily Search API is more reliable, faster, and legally safer. Build Playwright scrapers only for specific permitted sites.

3 agents for v1. Intent Analyzer + DB Retriever + Response Generator. Complexity should grow with validated user demand, not assumptions.

Validate with 10 users at Phase 1. Show a static Next.js mockup with hardcoded results. If users don't find it useful, the full architecture doesn't matter.

Auto-generate Zod from Pydantic. Use openapi-typescript to keep frontend/backend schemas in sync. Don't maintain two copies manually.